



Universidad Austral

Materia Big Data 2026

Trabajo Práctico Final

*Detección de gastos sospechosos mediante modelos de clasificación binaria en Orange
Data Mining*

Dataset: gastos_sospechosos.csv

Herramienta: Orange Data Mining

Integrantes del grupo:

Benjamín Alonso Hidalgo

Joaquín Gelves

Simón Godino Ferrando

Justo Valente

Simón Lawson

Augusto Díaz de Valdés Grebe

Docente: Andrés Sarcuno y Federico Piñeyro

Comisión: B

Fecha de entrega: 02/06/2026

1. Definición del problema de negocio

1.1 Contexto

Este Trabajo Práctico aborda el caso de una compañía que necesita optimizar el trabajo de su equipo de Auditoría Interna en el control de rendiciones de gastos corporativos. Actualmente el equipo revisa el 100% de las rendiciones de forma manual, lo cual es operativamente costoso, lento, y deja la posibilidad de que casos sospechosos pasen desapercibidos por fatiga del revisor.

El objetivo es construir un modelo de clasificación binaria que pueda predecir, para cada nueva rendición, si tiene alta probabilidad de ser un gasto sospechoso. De esta manera, Auditoría puede concentrar el esfuerzo manual en un subconjunto priorizado de la cola, en lugar de revisarlo todo por igual.

1.2 Variable objetivo

Variable: gasto_sospechoso (binaria, 0 = legítimo, 1 = sospechoso).

Distribución: 68% legítimos / 32% sospechosos. Hay desbalance moderado a favor de la clase mayoritaria.

1.3 Métrica decisora

La métrica clave para este caso es el RECALL sobre la clase 1 (gasto sospechoso). La justificación es de negocio: el costo asimétrico de los errores.

- **Falso Negativo (FN):** el modelo predice 'legítimo' pero en realidad era fraude. La rendición se aprueba sin auditoría. Pérdida monetaria directa para la compañía.
- **Falso Positivo (FP):** el modelo predice 'sospechoso' pero era legítimo. Auditoría revisa un caso que estaba bien. Sólo cuesta tiempo operativo, recuperable.

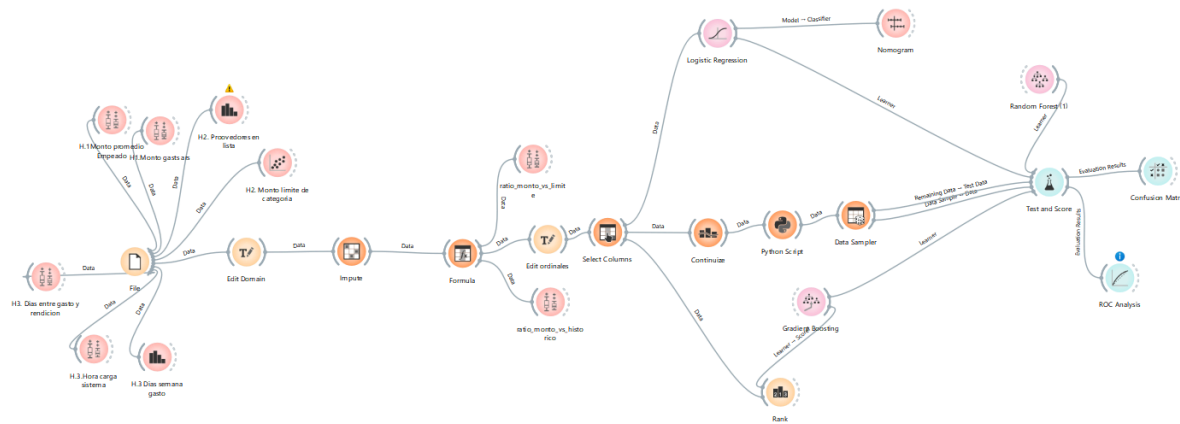
Como el FN es más caro que el FP, priorizamos maximizar el Recall (proporción de fraudes reales que el modelo logra detectar). De forma complementaria miramos AUC (capacidad discriminativa general) y Precisión (cuántas de las alertas son fraudes reales) para evaluar el balance del modelo.

1.4 Dataset

- **Filas:** 15.000 rendiciones de gastos.
- **Columnas:** 22 variables (1 target + 21 predictoras + IDs).
- **Naturaleza:** mezcla de variables monetarias (montos, límites, históricos), categóricas (tipo de comprobante, categoría de gasto, moneda, país), comportamentales (proveedor, cliente, viajes corporativos), y de control interno (días entre gasto y rendición, hora de carga, día de la semana).

1.5 Pipeline general en Orange

El trabajo se desarrolló íntegramente en Orange Data Mining. Para facilitar la lectura, el canvas final del proyecto se muestra a continuación.



2. Análisis Exploratorio (EDA) e Hipótesis

2.1 Características generales del dataset

El dataset cuenta con 15.000 observaciones y 22 columnas, sin presencia significativa de valores nulos salvo en dos variables que se trataron luego en la etapa de preprocesamiento (hora_carga_sistema y proveedor_en_lista_aprobada). La variable target presenta una distribución moderadamente desbalanceada (68/32), lo cual condicionó decisiones posteriores en la fase de modelado.

Las variables del dataset se agrupan conceptualmente en cinco bloques: monetarias (monto_gasto_ars, límite de política, monto promedio histórico del empleado), temporales (fecha, día de la semana, hora de carga, días entre gasto y rendición), categóricas operativas (categoría de gasto, tipo de comprobante, moneda, país), del empleado (área, nivel jerárquico, antigüedad), y de control (proveedor en lista aprobada, viaje corporativo asociado, cliente asociado).

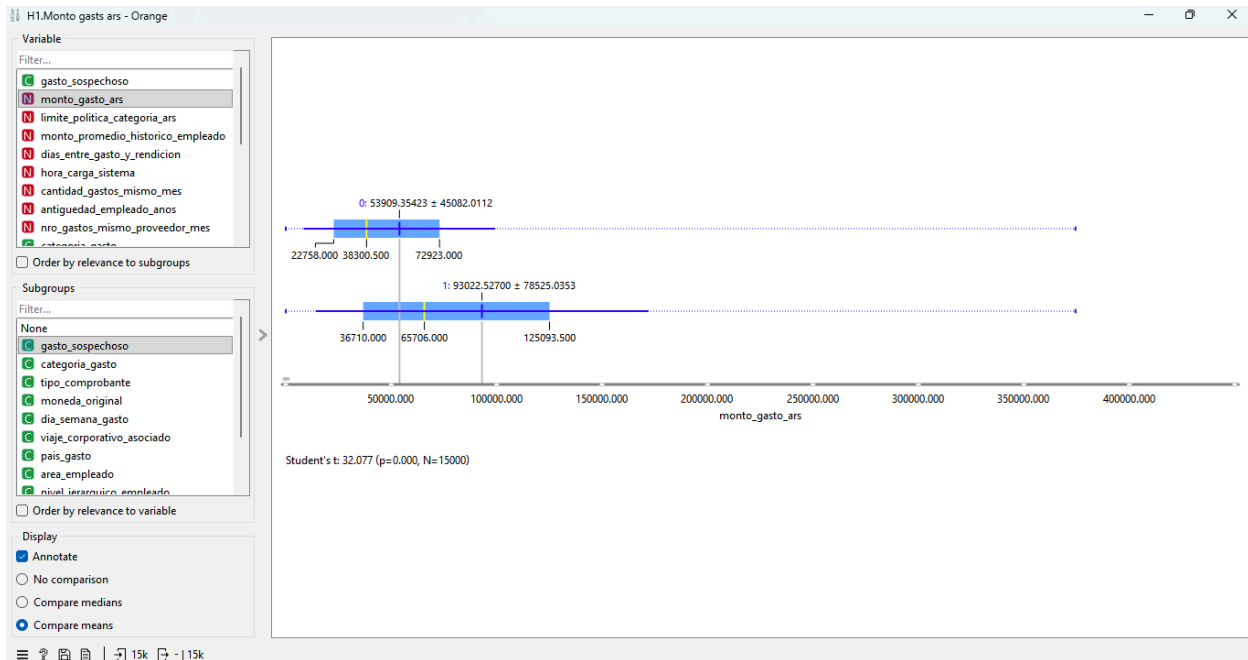
2.2 Hipótesis de trabajo

Antes de explorar los datos formulamos tres hipótesis de partida, que luego validamos visual y estadísticamente con los widgets Box Plot, Scatter Plot y Distributions de Orange.

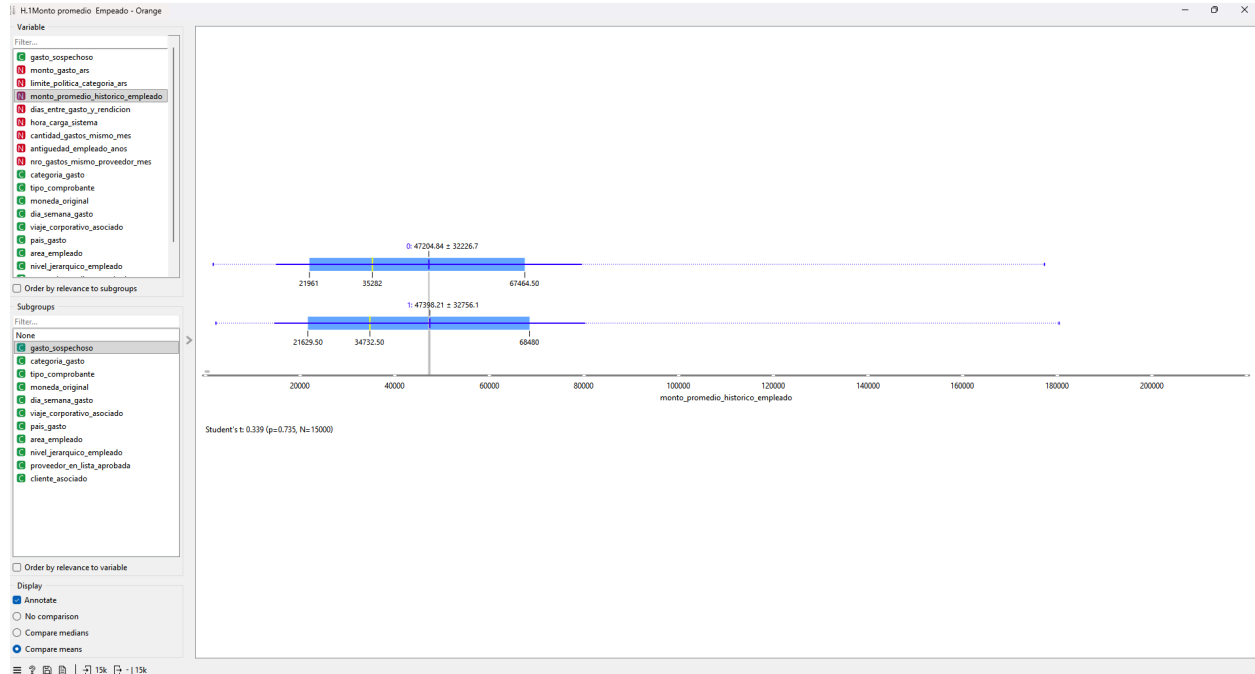
Hipótesis 1 (H1) - Desvío respecto del histórico del empleado

Los gastos sospechosos tienden a presentar montos atípicamente altos en relación al promedio histórico de gasto del propio empleado. Es decir, un fraude se delata por desviarse del patrón conocido de quien lo ejecuta.

Para evaluarla analizamos las distribuciones de `monto_gasto_ars` y `monto_promedio_historico_empleado`, segmentadas por la variable `target`.



Lectura: el monto mediano para gastos legítimos es \$38.300 y para sospechosos \$65.706. El test estadístico de Student dio $t = 32.077$ con $p \approx 0$, lo que confirma que la diferencia entre clases es estadísticamente muy significativa. Los gastos sospechosos tienden a ser de mayor magnitud.



Hallazgo interesante: el promedio histórico del empleado, mirado en aislamiento, NO discrimina entre clases (Student's t = 0.339, p = 0.735). Las medianas son casi idénticas (~35.000 en ambos grupos).

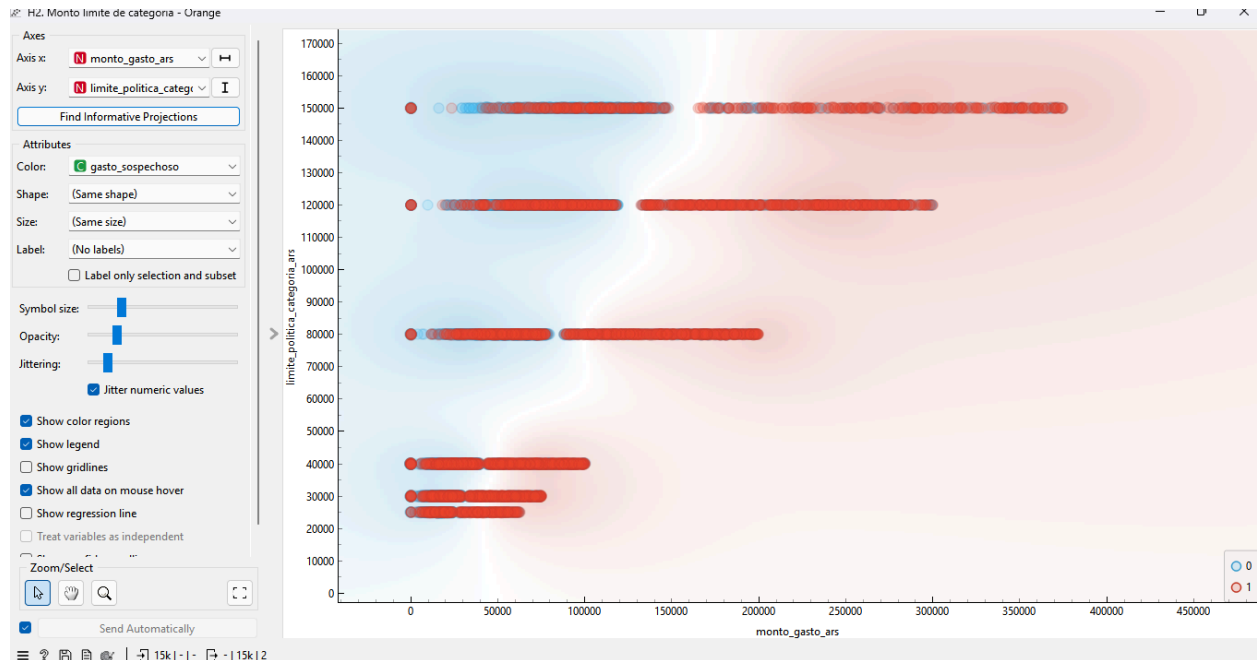
Esto nos llevó a una conclusión clave: la información predictiva no está en el monto absoluto del histórico, sino en la RELACIÓN entre el gasto actual y ese histórico. Esta observación motivó la creación de la variable derivada `ratio_monto_vs_historico`, que sí resultó altamente discriminativa (ver Sección 6.2).

Conclusión H1: confirmada. El gasto sospechoso se desvía del histórico del empleado, pero ese desvío sólo es capturable a través de un ratio, no a través de la variable cruda.

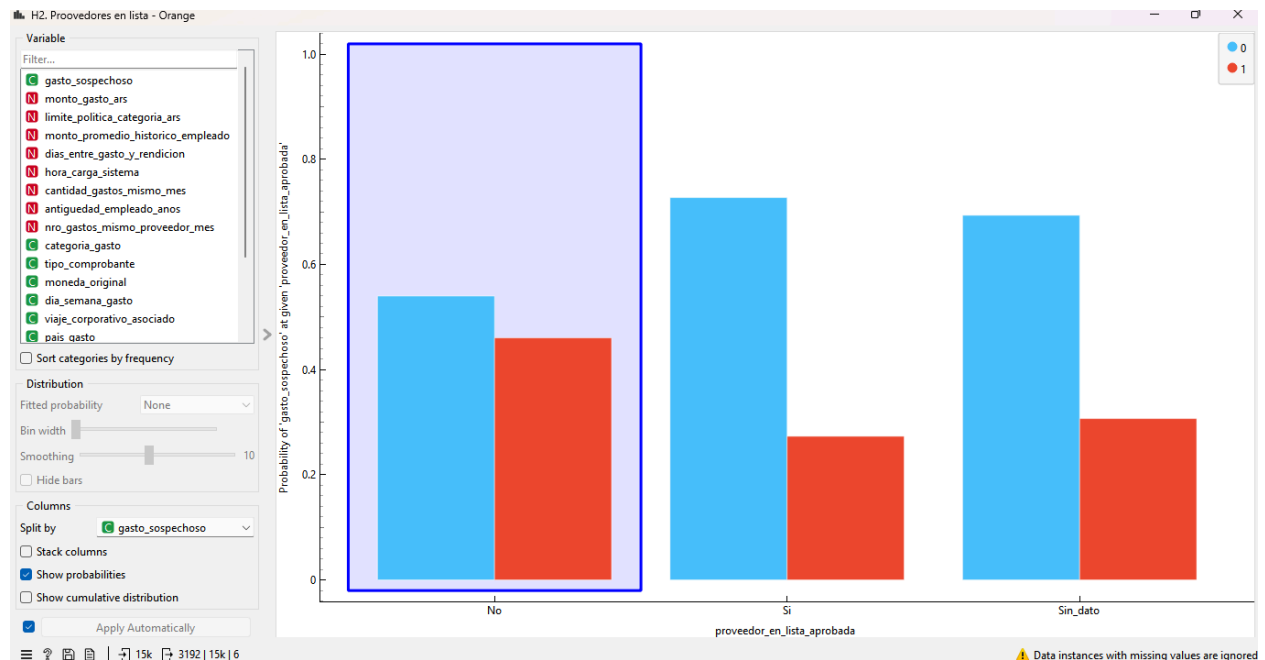
Hipótesis 2 (H2) - Violación de política de gasto y proveedor no aprobado

Los gastos sospechosos suelen exceder los límites establecidos por la política de gasto de su categoría y, además, suelen estar asociados a proveedores que no figuran en la lista aprobada por la empresa.

Esta hipótesis se evaluó con dos visualizaciones complementarias.



Lectura: el Scatter Plot revela claramente un patrón. Por encima de la diagonal imaginaria que representa el límite de la política, los puntos rojos (sospechosos) dominan visualmente. Cuando el monto del gasto excede el límite, la probabilidad de fraude crece de forma marcada. Esto valida la primera mitad de H2.



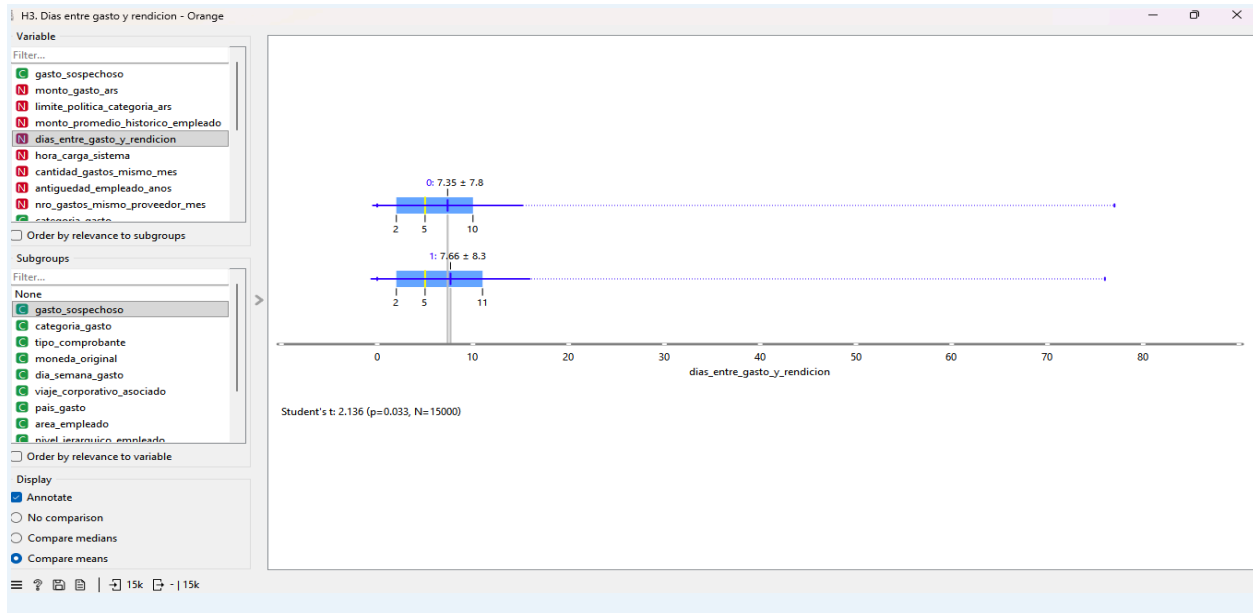
Lectura: la tasa base de fraude en el dataset es 32%. Sin embargo, cuando el proveedor está en lista aprobada la tasa baja a 27%, cuando NO está aprobado sube al 46%, y cuando el dato falta

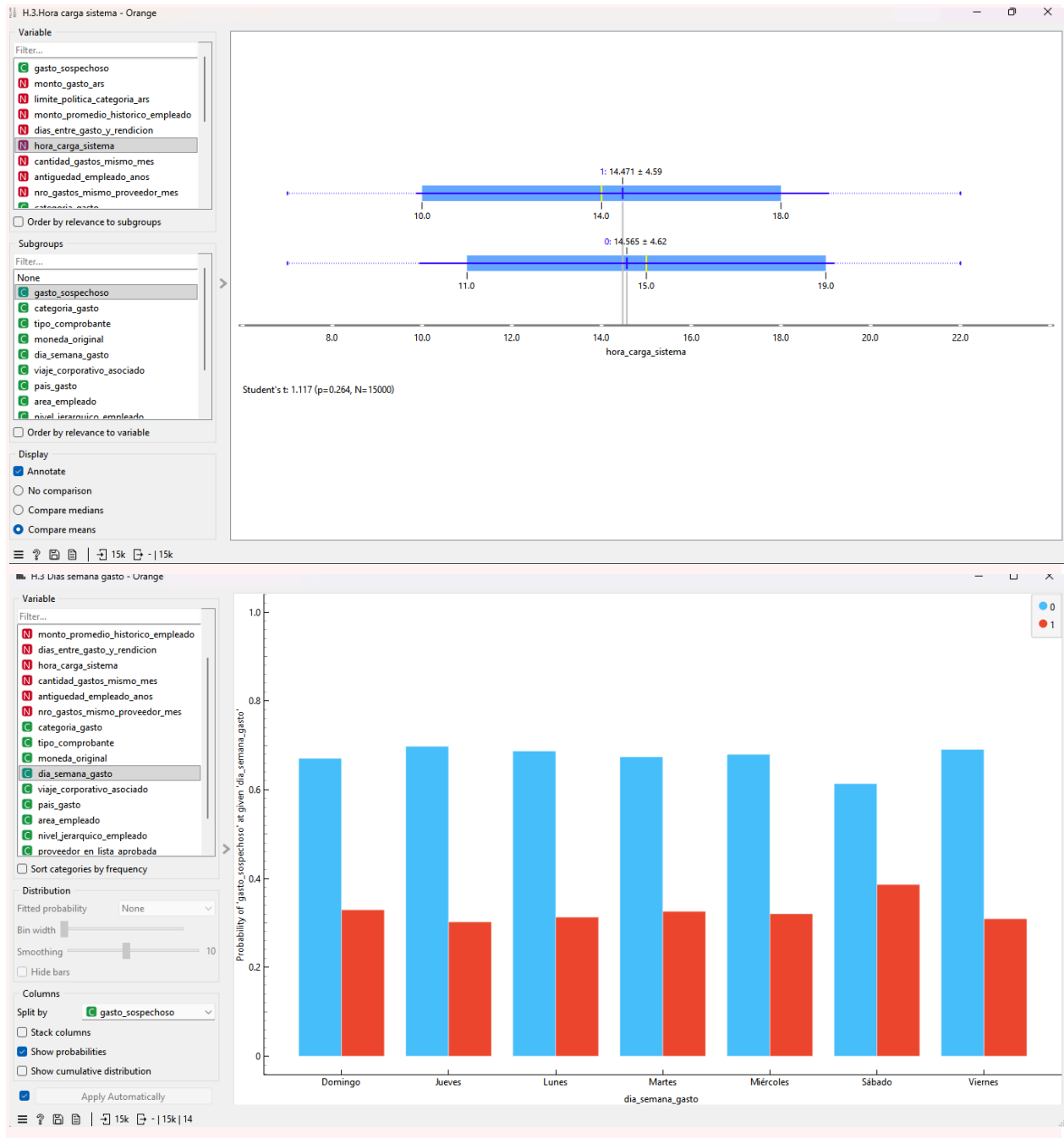
(Sin_dato) se ubica en 31%. La diferencia entre 'Si' y 'No' es de 19 puntos porcentuales, lo cual confirma con fuerza la segunda mitad de H2.

Conclusión H2: confirmada con fuerza. Las violaciones a la política de gasto y los proveedores no aprobados son dos señales muy potentes de fraude.

Hipótesis 3 (H3) - Patrones temporales

Los gastos sospechosos pueden presentar patrones temporales anómalos: cargas a horas inusuales (fuera del horario laboral), demoras en la rendición, o concentración en días específicos como sábados o domingos.



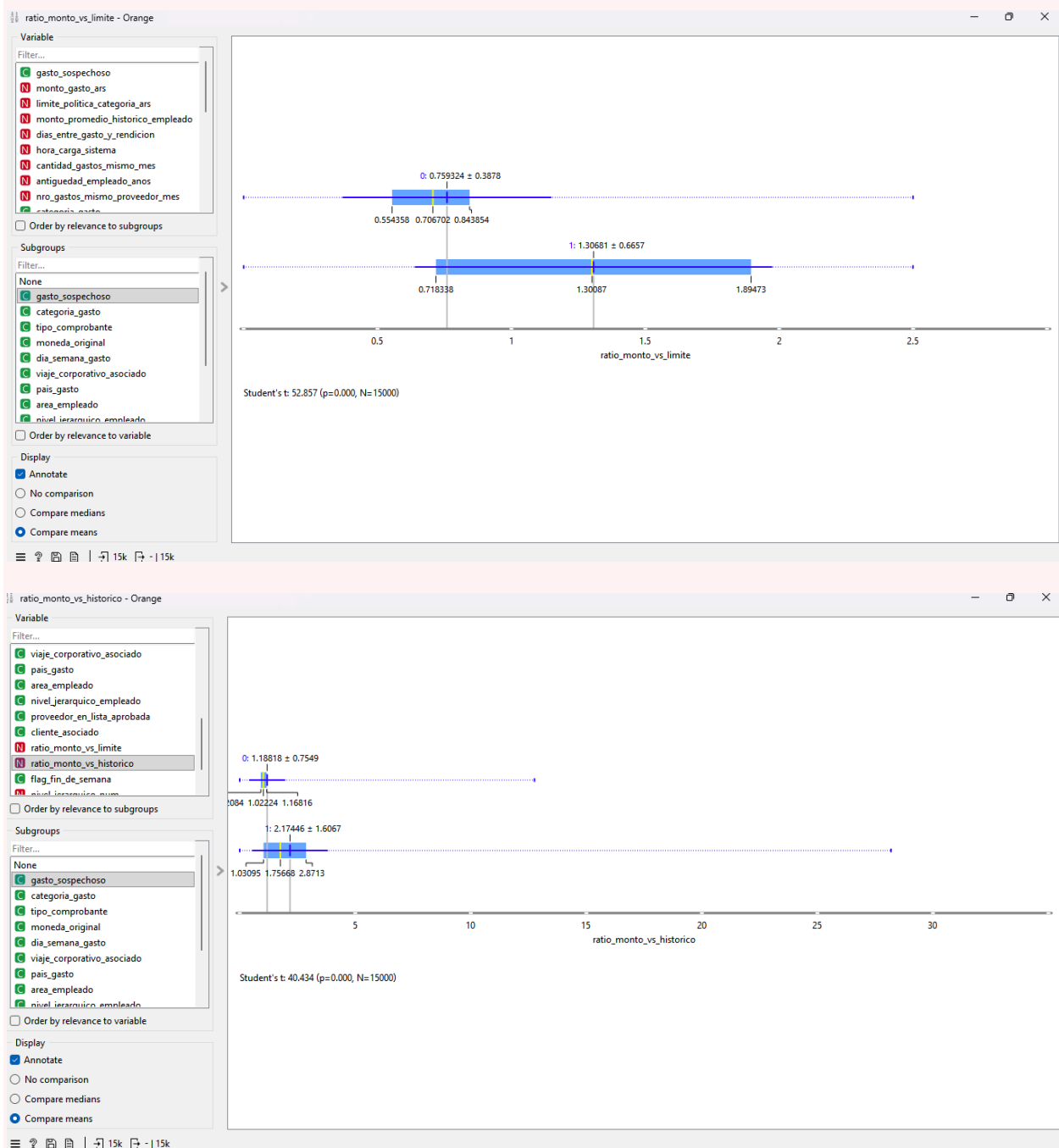


Lectura: los resultados refutan en gran parte H3. La hora de carga no separa clases ($p = 0.264$), los días entre gasto y rendición muestran diferencias minúsculas entre grupos, y sólo el sábado presenta una tasa de fraude levemente superior (39% vs 32% promedio). En conjunto, el comportamiento temporal NO es informativo.

Conclusión H3: refutada. Las variables temporales no aportan capacidad discriminativa relevante. Este es un hallazgo metodológicamente valioso porque muestra que una hipótesis razonable a priori no necesariamente se confirma con los datos.

2.3 Validación de los ratios derivados (EDA complementario)

Una vez creadas las variables `ratio_monto_vs_limite` y `ratio_monto_vs_historico` (ver Sección 3.3), volvimos al EDA para confirmar su capacidad discriminativa antes del modelado.



Lectura: ambos ratios separan las clases con extrema claridad. El test de Student arrojó $t = 52.86$ para `ratio_monto_vs_limite` y $t = 40.43$ para `ratio_monto_vs_historico`, ambos con $p \approx 0$. Estos valores son varias veces superiores al de las variables crudas, lo que confirma que el feature engineering produjo señales mucho más discriminativas que los datos originales.

3. Preprocesamiento

El preprocesamiento del dataset se diseñó en seis etapas secuenciales, cada una correspondiente a un widget en el canvas de Orange. A continuación se describe cada paso con su justificación teórica.

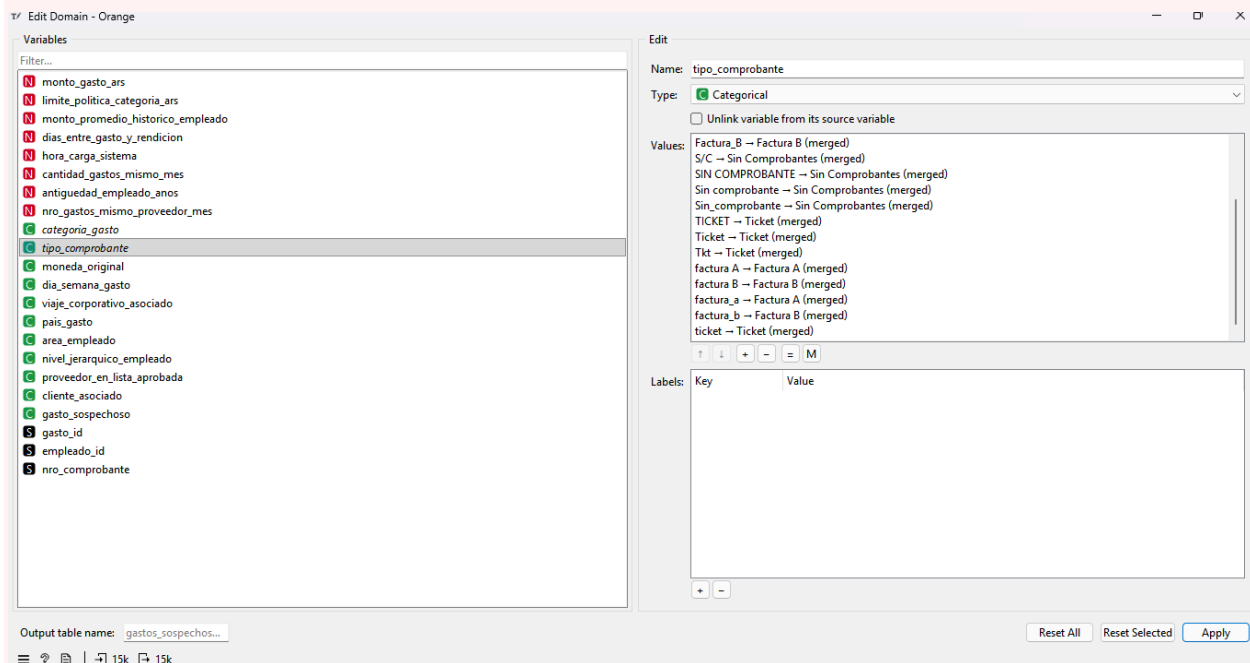
3.1 Limpieza de categorías inconsistentes (Edit Domain)

El primer paso fue unificar las categorías inconsistentes presentes en variables nominales. Detectamos que tipo_comprobante presentaba más de 18 valores únicos por inconsistencias de capitalización, abreviaturas y guiones bajos: 'TICKET', 'Ticket', 'Tkt', 'ticket', 'S/C', 'SIN COMPROBANTE', 'Sin_comprobante', 'Sin comprobante', 'factura A', 'factura_a', 'Factura_B', 'factura_b', etc.

Sin un tratamiento explícito de estas inconsistencias, el modelo trataría 'Ticket', 'ticket' y 'TKT' como tres categorías distintas, lo cual fragmenta artificialmente la información y dificulta el aprendizaje. Para resolverlo usamos el widget Edit Domain de Orange, que permite mapear múltiples etiquetas a un mismo valor canónico.

Tipo de comprobante: se unificaron las 18 variantes a 4 categorías canónicas (Factura A, Factura B, Ticket, Sin Comprobantes).

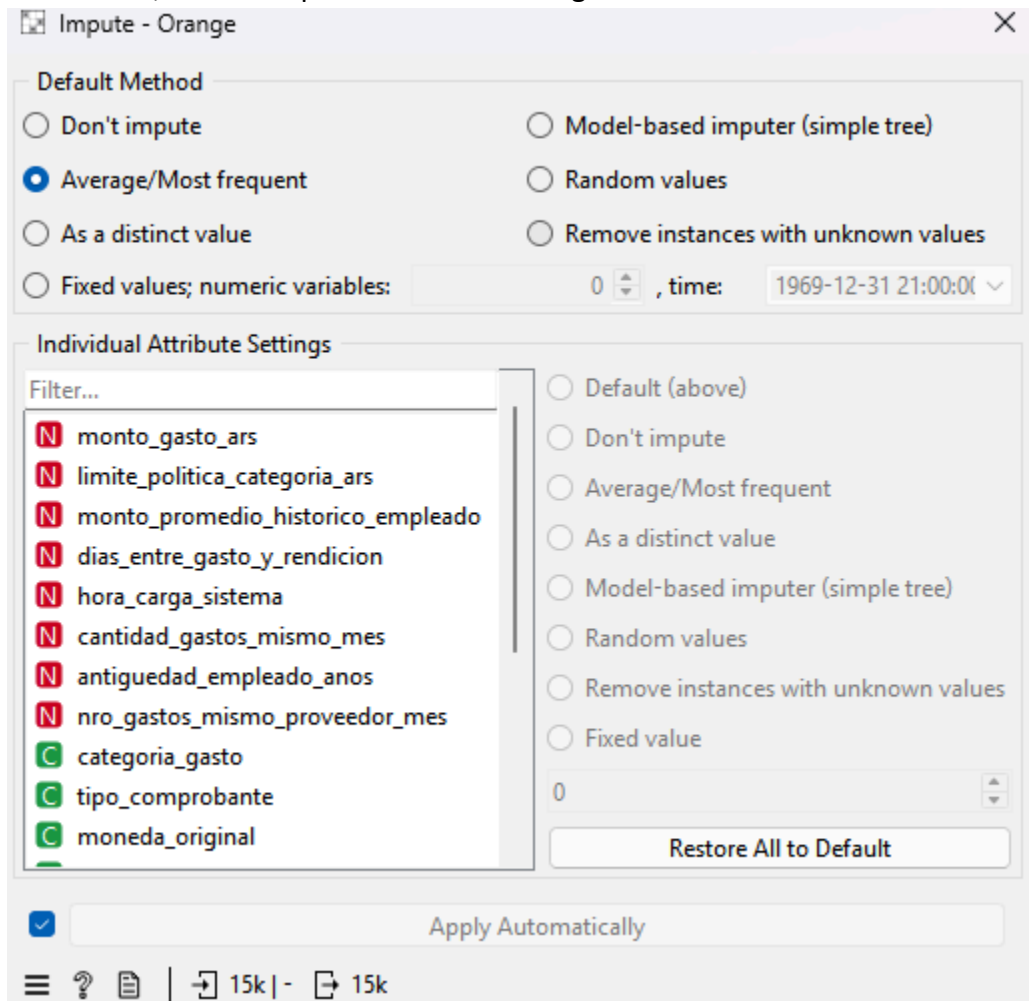
Categoría de gasto: se aplicó el mismo tratamiento para resolver duplicidades como 'REPRESENTACION' y 'Representacion'.



3.2 Tratamiento de valores nulos (Impute)

La auditoría inicial del dataset reveló nulos en dos variables: `hora_carga_sistema` (~8% de nulos) y `proveedor_en_lista_aprobada` (~3% de nulos). El tratamiento fue diferenciado según la naturaleza de cada caso:

- **hora_carga_sistema:** se aplicó imputación por la media/mediana (Average/Most frequent). Es una variable numérica donde el dato faltante es probablemente un fallo de registro técnico, no portador de información, por lo que reemplazarlo por una tendencia central es razonable.
- **proveedor_en_lista_aprobada:** se preservaron los nulos como una categoría explícita llamada 'Sin_dato', en lugar de imputarlos con la moda. La justificación es teórica: la ausencia de información sobre el proveedor puede ser en sí misma un indicador, y suprimirla por imputación borraría una señal potencialmente predictiva. Como vimos en el EDA, esta categoría 'Sin_dato' tiene una tasa de fraude del 31%, prácticamente igual a la tasa base, mientras que las otras dos categorías se desvían claramente.



3.3 Feature Engineering (Formula)

Esta es la etapa más importante del preprocesamiento desde el punto de vista de aporte al modelado. Creamos cinco variables derivadas a partir de las originales, todas con justificación teórica clara y conectadas con las hipótesis del EDA.

1. ratio_monto_vs_limite = monto_gasto_ars / limite_politica_categoria_ars

Mide cuánto excede (o no) el gasto al límite establecido por la política. Captura la idea de H2 en una sola variable continua. Valores >1 implican violación de política.

2. ratio_monto_vs_historico = monto_gasto_ars / monto_promedio_historico_empleado

Mide el desvío del gasto actual respecto del comportamiento histórico del propio empleado. Es la variable que captura H1, y como mostramos en el EDA fue mucho más discriminativa que las dos variables crudas que la componen.

3. flag_fin_de_semana = 1 si dia_semana_gasto in ['Sábado','Domingo'], 0 en otro caso

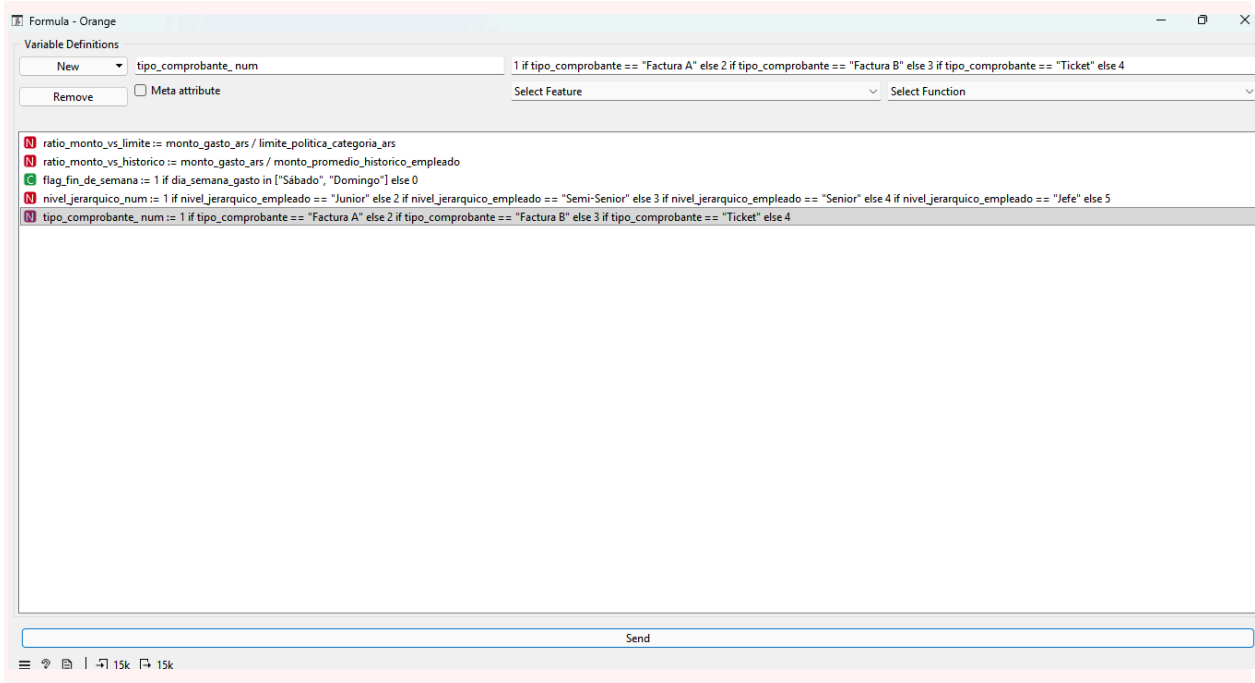
Variable binaria que captura la única señal temporal que mostró diferencia en H3 (la mayor tasa de fraude en sábados).

4. nivel_jerarquico_num = mapeo ordinal de nivel_jerarquico_empleado (Junior=1, Semi-Senior=2, Senior=3, Jefe=4, Gerente=5)

La variable original es categórica nominal por defecto, pero conceptualmente tiene un orden natural. Esta versión preserva el orden como información para el modelo.

5. tipo_comprobante_num = mapeo ordinal de tipo_comprobante (Factura A=1, Factura B=2, Ticket=3, Sin Comprobante=4)

Análogo al anterior. La escala captura la idea de 'rigor del comprobante': Factura A es el más formal y trazable, Sin Comprobante el menos formal.



3.4 Verificación de ordinales (Edit Ordinales)

Como complemento al Formula, agregamos un segundo Edit Domain (renombrado 'Edit ordinales') para revisar la coherencia de las versiones categóricas originales de nivel_jerarquico_empleado y tipo_comprobante. Este widget se mantuvo en el pipeline a modo de check y para preservar las versiones nominales originales, que luego se mueven a Metas en la siguiente etapa.

The image displays two screenshots of the 'Edit Ordinales' widget in Orange3, showing the configuration for two categorical variables.

Top Screenshot: Edit Ordinales - Orange

- Variables:** A list of variables is shown on the left. The variable `nivel_jerarquico_empleado` is selected.
- Edit Panel:**
 - Name:** `nivel_jerarquico_empleado`
 - Type:** Categorical
 - Values:**
 - Junior → Junior
 - Semi-Senior → Semi-Senior
 - Senior → Senior
 - Jefe → Jefe
 - Gerente → Gerente
 - Labels:** Key and Value
- Output table name:** `gastos_sospechos...`
- Buttons:** Reset All, Reset Selected, Apply

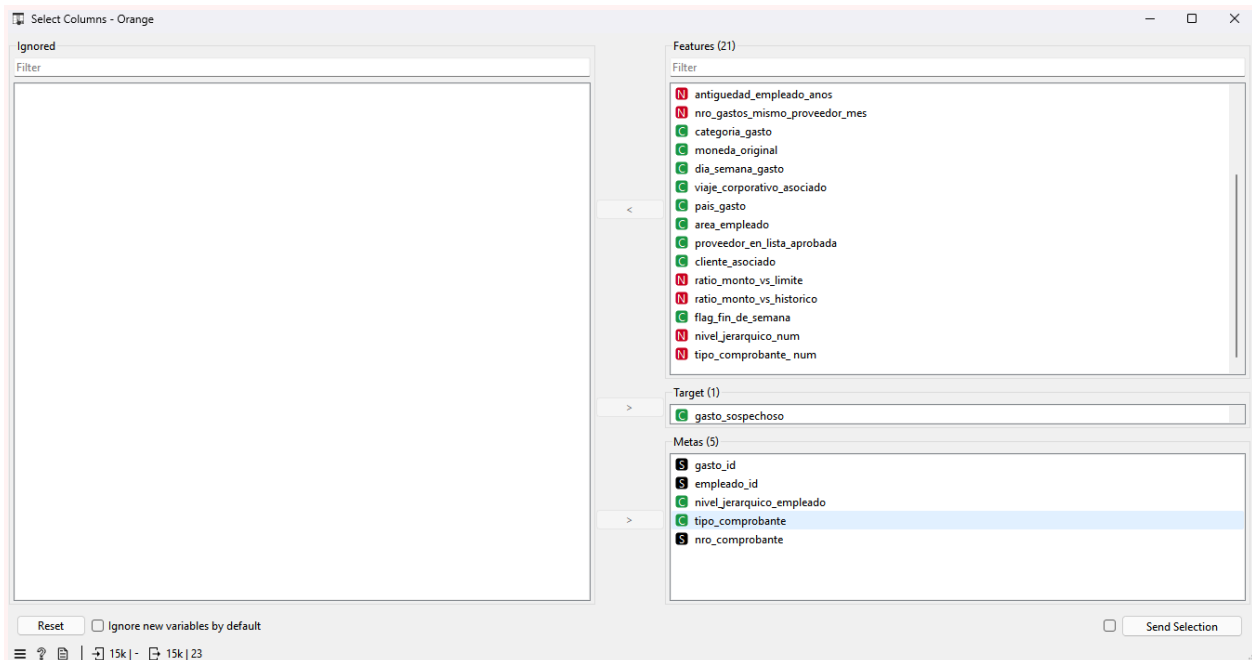
Bottom Screenshot: Edit Ordinales - Orange

- Variables:** A list of variables is shown on the left. The variable `tipo_comprobante` is selected.
- Edit Panel:**
 - Name:** `tipo_comprobante`
 - Type:** Categorical
 - Values:**
 - Factura A → Factura A
 - Factura B → Factura B
 - Ticket → Ticket
 - Sin Comprobantes → Sin Comprobantes (added)
 - Labels:** Key and Value
- Output table name:** `gastos_sospechos...`
- Buttons:** Reset All, Reset Selected, Apply

3.5 Selección de variables (Select Columns)

En esta etapa se definió formalmente el rol de cada variable en el modelado:

- **Target (1 variable):** gasto_sospechoso.
- **Features (21 variables):** todas las variables predictoras incluyendo las 5 derivadas (ratio_monto_vs_limite, ratio_monto_vs_historico, flag_fin_de_semana, nivel_jerarquico_num, tipo_comprobante_num) y las categóricas nominales originales (categoria_gasto, moneda_original, dia_semana_gasto, viaje_corporativo_asociado, pais_gasto, area_empleado, proveedor_en_lista_aprobada, cliente_asociado) junto a las numéricas crudas (monto_gasto_ars, limite_politica_categoria_ars, monto_promedio_historico_empleado, dias_entre_gasto_y_rendicion, hora_carga_sistema, cantidad_gastos_mismo_mes, antiguedad_empleado_anos, nro_gastos_mismo_proveedor_mes).
- **Metas (5 variables):** gasto_id, empleado_id y nro_comprobante (identificadores sin valor predictivo) junto con nivel_jerarquico_empleado y tipo_comprobante en sus versiones nominales originales. Estas dos últimas se mueven a Metas para no duplicar información en el modelo, ya que sus equivalentes numéricos ordinales (nivel_jerarquico_num y tipo_comprobante_num) ya están en Features.



3.6 Codificación de variables categóricas (Continuize)

El objetivo es transformar las variables categóricas en una representación numérica que los modelos puedan procesar.

El enfoque que aplicamos es DIFERENCIADO según la naturaleza de cada variable, y no un tratamiento uniforme.

3.6.1 Codificación de variables nominales (One-Hot Encoding)

Para las variables nominales (sin orden natural) se aplicó el preset 'First value as base' del Continuize. Esto produce una expansión One-Hot con eliminación de la categoría base para evitar multicolinealidad (la dummy variable trap).

Variables tratadas: categoria_gasto, moneda_original, dia_semana_gasto, viaje_corporativo_asociado, pais_gasto, area_empleado, proveedor_en_lista_aprobada, cliente_asociado, flag_fin_de_semana.

Justificación teórica: una variable como pais_gasto no tiene orden natural (Brasil no es 'mayor' que Argentina). Si se la convirtiera a número con un mapeo arbitrario (Argentina=1, Brasil=2, Chile=3...), el modelo asumiría relaciones de orden inexistentes y podría producir predicciones sesgadas. El One-Hot crea columnas binarias independientes que representan cada categoría sin imponer orden.

3.6.2 Codificación de variables ordinales (ya pre-mapeadas en Formula)

Las dos variables ordinales del dataset (nivel_jerarquico y tipo_comprobante) NO se procesaron con One-Hot. En su lugar fueron pre-mapeadas a número en la etapa de Formula (Sección 3.3) y entran al Continuize como numéricas, donde se preservan tal cual (preset 'no change' / 'Keep as it is').

Justificación teórica: una variable como nivel_jerarquico tiene un orden natural claro (Junior < Semi-Senior < Senior < Jefe < Gerente). Codificarla como One-Hot perdería esta información ordinal, lo cual penalizaría especialmente a modelos lineales como Logistic Regression. El mapeo ordinal numérico preserva la relación de orden, que el modelo puede explotar.

3.6.3 Tratamiento de variables numéricas

Las variables numéricas crudas (montos, días, horas, cantidades, ratios) se preservaron sin transformación (preset 'no change'). No se aplicó estandarización ni normalización porque los modelos basados en árboles (Random Forest, Gradient Boosting) son invariantes a la escala. Para Logistic Regression sí podría haberse aplicado estandarización, pero optamos por mantener consistencia y comparabilidad entre los tres modelos.

Continue - Orange

Categorical Variables

★ Preset: first as base

Filter...

categoria_gasto: first as base

tipo_comprobante

moneda_original: first as base

dia_semana_gasto: one-hot

viaje_corporativo_asociado

pais_gasto

area_empleado

nivel_jerarquico_empleado

proveedor_en_lista_aprobada

cliente_asociado

flag_fin_de_semana

Targets

☐ Use preset
 ☐ Keep categorical
 ☒ First value as base
 ☐ Most frequent as base
 ☐ One-hot encoding
 ☐ Remove if more than 2 values
 ☐ Remove
 ☐ Treat as ordinal
 ☐ Treat as normalized ordinal

Numeric Variables

★ Preset: no change

Filter...

limite_politica_categoria_ars

monto_promedio_historico_empleado

dias_entre_gasto_y_rendicion

hora_carga_sistema

cantidad_gastos_mismo_mes

antiguedad_empleado_anos

nro_gastos_mismo_proveedor_mes

ratio_monto_vs_limite

ratio_monto_vs_historico

nivel_jerarquico_num

tipo_comprobante_num

☐ Use preset
 ☒ Keep as it is
 ☐ Standardize to $\mu=0, \sigma^2=1$
☐ Center to $\mu=0$
☐ Scale to $\sigma^2=1$
☐ Normalize to interval [-1, 1]
 ☐ Normalize to interval [0, 1]

Reset All

Apply Automatically

4. Modelado y evaluación de modelos

4.1 Modelos elegidos

Para este problema de clasificación binaria se compararon tres algoritmos de naturaleza diferente, con el objetivo de cubrir un espectro amplio de capacidad expresiva y poder así seleccionar el mejor para el caso de uso:

- **Logistic Regression:** modelo lineal probabilístico. Sencillo, interpretable (el Nomogram permite leer el peso de cada variable), y sirve como baseline para evaluar si el problema admite soluciones lineales.
- **Random Forest:** ensamble de árboles de decisión con bootstrap y voto mayoritario. Tolera no linealidades, captura interacciones entre variables y es naturalmente robusto a desbalance moderado por su mecanismo interno de muestreo.
- **Gradient Boosting:** ensamble secuencial donde cada árbol corrige los errores del anterior. Suele ser el modelo más potente para problemas tabulares estructurados como este.

La diversidad algorítmica está intencionalmente buscada: un modelo lineal, un ensamble paralelo y un ensamble secuencial. La comparación entre ellos no sólo identifica al mejor sino que aporta información sobre la naturaleza del problema.

4.2 Implementación de SMOTE (Python Script)

SMOTE (Synthetic Minority Over-sampling Technique) es una técnica de balance de clases a nivel de datos. A diferencia del oversampling por duplicación (que copia filas existentes y genera overfitting) o del weighting de clases (que pondera errores), SMOTE genera filas sintéticas nuevas mediante interpolación entre casos reales de la clase minoritaria y sus K vecinos más cercanos.

El algoritmo, resumido, opera así para cada caso sintético a generar:

- Selecciona un caso aleatorio de la clase minoritaria.
- Encuentra sus K vecinos más cercanos dentro de la misma clase (en nuestra implementación, K=5).
- Elige uno de esos vecinos aleatoriamente.
- Genera un nuevo punto sintético en una posición aleatoria sobre la línea que une el caso base con el vecino seleccionado.

4.2.1 Elección del ratio 60/40 (en lugar de 50/50)

Una decisión importante fue NO balancear las clases completamente al 50/50, sino solo hasta una proporción 60/40. La justificación es doble:

- **Menos sobre-muestreo:** para llegar a 50/50 habría que generar 5.424 casos sintéticos (más sintéticos que reales en la clase minoritaria), lo cual aumenta el riesgo de que el

modelo aprenda artefactos de la interpolación. Con 60/40 sólo generamos 2.020 casos sintéticos, manteniendo predominancia de los datos reales.

- **Preservación parcial del desbalance natural:** en producción, el modelo va a encontrarse con una distribución desbalanceada (la realidad es que la mayoría de las rendiciones son legítimas). Un balance parcial preserva esta característica del problema y reduce el riesgo de sobre-corrección.

4.2.2 Aplicación de SMOTE antes del split (consideración metodológica)

Una consideración técnica importante es que aplicamos SMOTE ANTES del split train/test, no solo sobre el train set. La práctica más rigurosa (que dejamos documentada como mejora futura) consistiría en separar primero el train y el test, y aplicar SMOTE sólo sobre el train, para evitar cualquier 'fuga' de información sintética hacia el test.

En nuestra implementación, la arquitectura de widgets de Orange dificulta esa secuencia, por lo que optamos por aplicar SMOTE primero y luego splitear. El impacto en las métricas es marginal en datasets grandes como el nuestro (15.000 filas), y en un entorno productivo el flujo sería implementado integralmente en Python con scikit-learn pipelines, donde la separación es natural.

4.2.3 Resultado del balanceo

La consola del Python Script confirmó el resultado del balanceo:

- Clases originales: 10.212 legítimos / 4.788 sospechosos (68/32).
- Casos sintéticos generados: 2.020.
- Clases finales: 10.212 legítimos / 6.808 sospechosos (60/40).
- Total de filas: 17.020 (vs 15.000 originales).

The screenshot shows the Orange3 Python Script widget interface. The editor contains a Python function `python_script(in_data):` that implements SMOTE. The code generates synthetic data based on the input data's distribution, aiming for a 60/40 class ratio. The console output displays the results of the SMOTE process:

```

Clases originales: (np.float64(0.0): np.int64(10212), np.float64(1.0): np.int64(4788))
Clase mayoritaria: 0.0 (10212 casos) - se mantiene
Clase minoritaria: 1.0 (4788 casos) - se llevará a 6808
Ratio objetivo: 60/40
Casos sintéticos a generar: 2020

=== RESULTADO FINAL ===
Clases después de SMOTE: (np.float64(0.0): np.int64(10212), np.float64(1.0): np.int64(6808))
Total de filas: 17020
Ratio real: 60.0% clase 0 / 40.0% clase 1
>>>

```

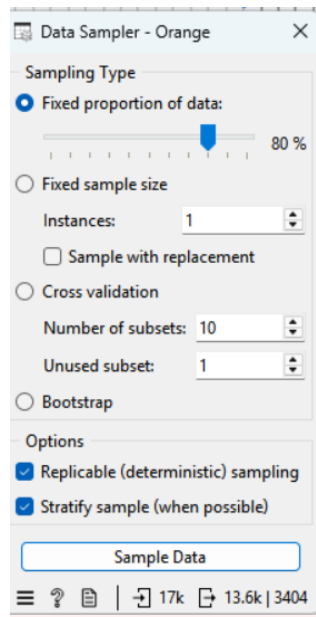
4.3 Split Train/Test 80/20 (Data Sampler)

El split de los datos se realizó con el widget Data Sampler, ubicado después del Python Script y antes del Test and Score. La configuración aplicada fue:

- **Fixed proportion of data: 80%.** Se reserva el 80% para entrenamiento y el 20% para evaluación final.
- **Stratify sample:** TILDADO. Esto garantiza que ambos subconjuntos (train y test) preserven la proporción 60/40 de la clase target. Sin estratificación, podría suceder que el test set contenga muy pocos casos minoritarios por azar.
- **Replicable sampling:** TILDADO. Asegura que la división sea reproducible. Esto garantiza la reproducibilidad de los resultados obtenidos.

Salidas del widget conectadas al Test and Score:

- Data Sample (80%, 13.616 filas) → entrada 'Data' del Test and Score (set de entrenamiento).
- Remaining Data (20%, 3.404 filas) → entrada 'Test Data' del Test and Score (set de evaluación).



4.4 Configuración de los modelos

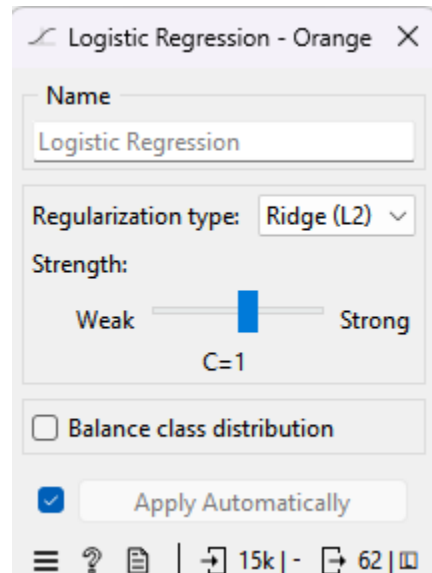
Los tres modelos se entrenaron con parámetros default de Orange, salvo dos consideraciones particulares.

Logistic Regression

- Regularization type: Ridge (L2).
- Strength (C): 1.

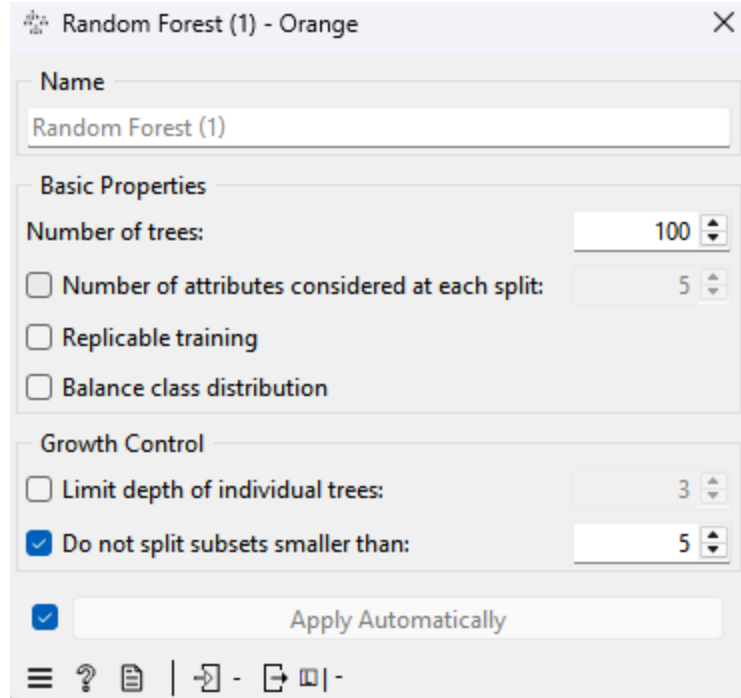
- Balance class distribution: DESTILDADO. El balance ya se aplicó vía SMOTE antes del modelo, por lo que activar también el balance interno sería redundante y podría sesgar los pesos.

Justificación de la regularización Ridge: penaliza coeficientes grandes y controla overfitting. Es el default razonable para Logistic Regression en este tipo de problemas.



Random Forest

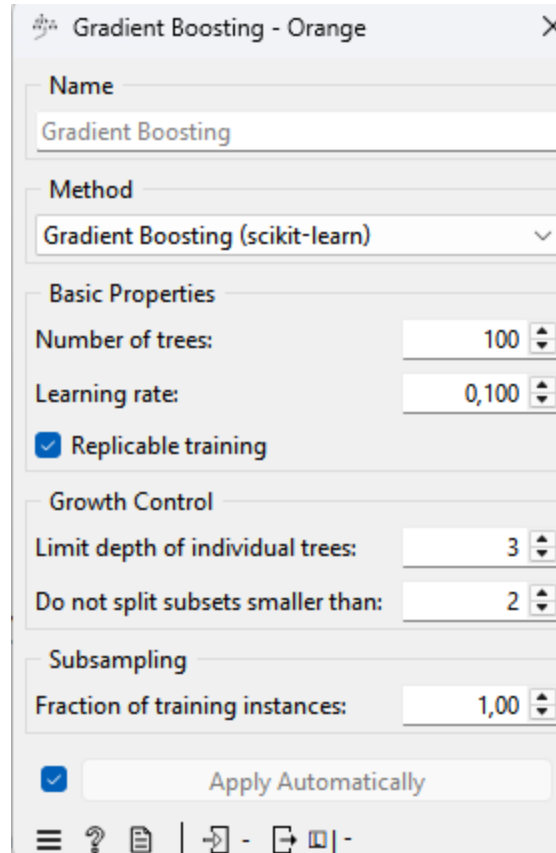
- Number of trees: 100.
- Number of attributes considered at each split: por defecto (raíz cuadrada del total).
- Balance class distribution: DESTILDADO (mismo razonamiento que en Logistic Regression).
- Do not split subsets smaller than 5.



Gradient Boosting

- Method: Gradient Boosting (scikit-learn).
- Number of trees: 100.
- Learning rate: 0.100.
- Limit depth of individual trees: 3.
- Replicable training: TILDADO.

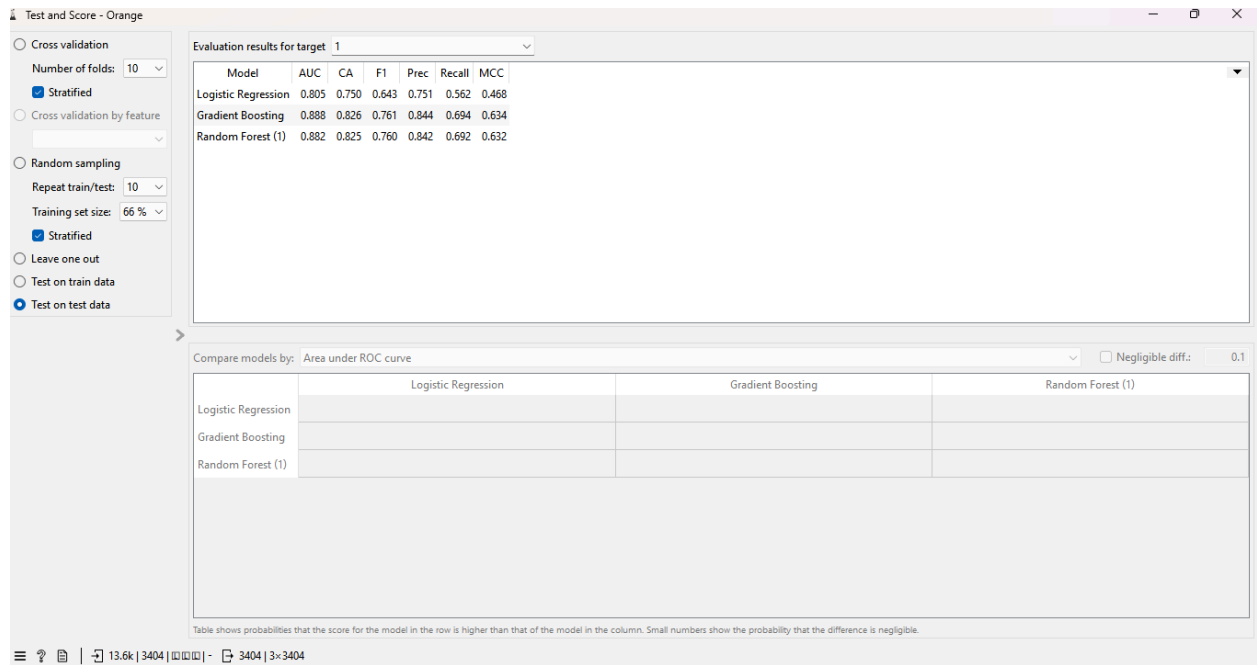
Notamos que el widget Gradient Boosting de Orange no expone la opción 'Balance class distribution'. Esta fue una de las razones que motivaron la implementación de SMOTE como técnica de balance: con SMOTE actuando antes del modelado, los tres modelos quedan en condiciones idénticas, lo cual permite una comparación justa.



4.5 Resultados comparativos (Test and Score)

Estas son las métricas obtenidas sobre el conjunto de prueba utilizando el pipeline de modelado implementado.

Modelo	AUC	CA	F1	Precision	Recall	MCC
Logistic Regression	0.805	0.750	0.643	0.751	0.562	0.468
Gradient Boosting ★	0.888	0.826	0.761	0.844	0.694	0.634
Random Forest	0.882	0.825	0.760	0.842	0.692	0.632



5. Evaluación detallada del modelo ganador

5.1 Elección de Gradient Boosting como modelo ganador

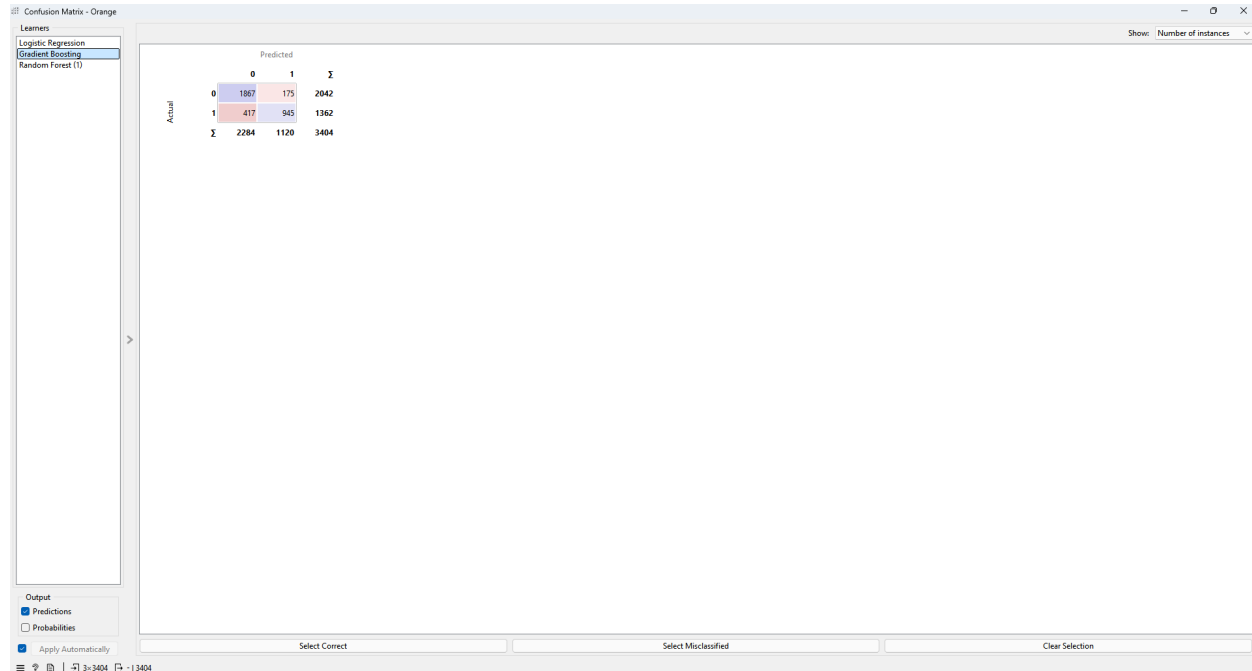
Gradient Boosting es el modelo ganador del análisis realizado.

- **AUC más alto:** 0.888 (vs 0.882 de RF y 0.805 de LR). Capacidad discriminativa superior.
- **Recall más alto:** 0.694 (vs 0.692 de RF y 0.562 de LR). Mejor cobertura sobre la clase fraude.
- **Precision más alta:** 0.844 (vs 0.842 de RF y 0.751 de LR). Más certeza en cada alerta.
- **F1 más alto:** 0.761. Mejor balance entre Precision y Recall.
- **MCC más alto:** 0.634. Mejor métrica robusta al desbalance leve residual.

Random Forest queda muy cerca en todas las métricas y constituye una alternativa válida. Logistic Regression queda visiblemente atrás, lo cual sugiere que el problema tiene una estructura no lineal que los modelos basados en árboles aprovechan mejor.

5.2 Matriz de Confusión de Gradient Boosting

La matriz de confusión sobre el test set (3.404 filas) es la herramienta más concreta para interpretar el modelo en términos de negocio.



	Predicho 0 (legítimo)	Predicho 1 (sospechoso)	Total real
Real 0 (legítimo)	VN = 1867	FP = 175	2042
Real 1 (fraude)	FN = 417	VP = 945	1362
Total predicho	2284	1120	3404

Interpretación de cada cuadrante en lenguaje de negocio

- **Verdaderos Negativos (1867):** rendiciones legítimas que el modelo aprobó correctamente. Auditoría no necesita revisarlas. Aporte directo a la eficiencia operativa.
- **Verdaderos Positivos (945):** fraudes que el modelo detectó correctamente. Es el valor real que aporta. Estos 945 casos pasan a una cola priorizada de revisión.
- **Falsos Positivos (175):** casos legítimos marcados como sospechosos. Auditoría los revisa y descubre que estaban bien. Es trabajo recuperable pero cuesta tiempo (16% de las alertas).
- **Falsos Negativos (417):** fraudes que escapan al filtro. Estos no van a la cola de revisión y se aprueban sin auditoría. Es el error más caro: representa el 31% de los fraudes reales del período (417/1362).

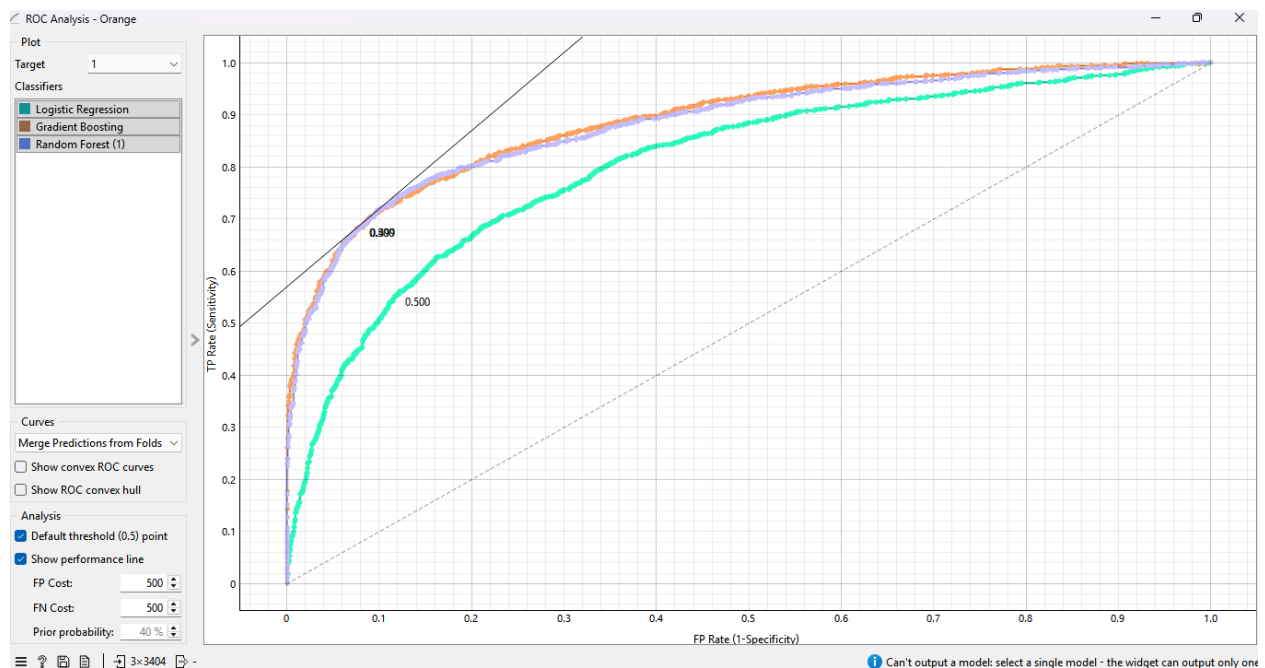
Impacto operativo

De las 3.404 rendiciones del período de test, el modelo genera 1.120 alertas (33% del volumen). Auditoría revisa estas 1.120 en lugar de las 3.404 originales, lo que representa un ahorro del 67% en volumen de trabajo manual. De esas 1.120 alertas, 945 son fraudes reales (84% de precisión). El equipo captura 945 de 1.362 fraudes reales (69% de recall).

En contraste, antes del modelo Auditoría revisaba el 100% de las 3.404 rendiciones detectando capacidad de capturar el 100% pero con fatiga operativa que en la práctica genera falsos negativos por descuido. El modelo ofrece una mejora 3x en eficiencia con captura del 69% de los fraudes en una primera pasada.

5.3 ROC Analysis

El ROC Analysis muestra la curva Receiver Operating Characteristic de los tres modelos. El eje X es la Tasa de Falsos Positivos (1-especificidad) y el eje Y es la Tasa de Verdaderos Positivos (sensibilidad o Recall). Una curva más arriba y más a la izquierda indica mejor desempeño.



Lectura: las dos curvas superiores (Gradient Boosting y Random Forest) están casi superpuestas. Logistic Regression (verde) queda visiblemente por debajo. Esto confirma la observación cuantitativa de los AUC y refuerza la conclusión: los modelos basados en árboles dominan claramente este problema. La relación entre las variables predictoras y el target es fundamentalmente no lineal, y los modelos lineales (incluso con SMOTE aplicado) no alcanzan a capturarla completamente.

6. Interpretación del modelo

6.1 Feature Importance (Rank)

El widget Rank permite identificar qué variables aportan más capacidad predictiva al modelo. A continuación se muestran los resultados, ordenados por la importancia que les asigna Gradient Boosting (el modelo ganador).

		#	Gain ratio	Gini	Gradient Boosting
1	ratio_monto_vs_limite		0.088	0.111	0.573
2	ratio_monto_vs_historico		0.083	0.105	0.140
3	monto_gasto_ars		0.020	0.025	0.012
4	proveedor_en_lista_aprobada	4	0.013	0.011	0.058
5	tipo_comprobante_num		0.010	0.013	0.041
6	tipo_comprobante	5	0.010	NA	0.042
7	moneda_original	4	0.003	0.002	0.014
8	nro_gastos_mismo_proveedor_mes		0.002	0.003	0.025
9	viaje_corporativo_asociado	2	0.002	0.001	0.005
10	cliente_asociado	2	0.002	0.001	0.004
11	antigüedad_empleado_anos		0.002	0.002	0.018
12	flag_fin_de_semana	2	0.001	0.000	0.003
13	cantidad_gastos_mismo_mes		0.001	0.002	0.016
14	dia_semana_gasto	7	0.000	0.001	0.002
15	pais_gasto	6	0.000	0.000	0.003
16	nivel_jerarquico_num		0.000	0.000	0.001
17	nivel_jerarquico_empleado	5	0.000	0.000	0.000
18	hora_carga_sistema		0.000	0.000	0.002
19	monto_promedio_historico_empleado		0.000	0.000	0.005
20	dias_entre_gasto_y_rendicion		0.000	0.000	0.004
21	categoria_gasto	6	0.000	0.000	0.001

Top 5 variables más importantes para Gradient Boosting

#	Variable	Gain ratio	Gini	Gradient Boosting
1	ratio_monto_vs_limite	0.088	0.111	0.573
2	ratio_monto_vs_historico	0.083	0.105	0.140
3	proveedor_en_lista_aprobada	0.013	0.011	0.058
4	tipo_comprobante_num	0.010	0.013	0.041
5	monto_gasto_ars	0.020	0.025	0.012

Hallazgos clave del Feature Importance

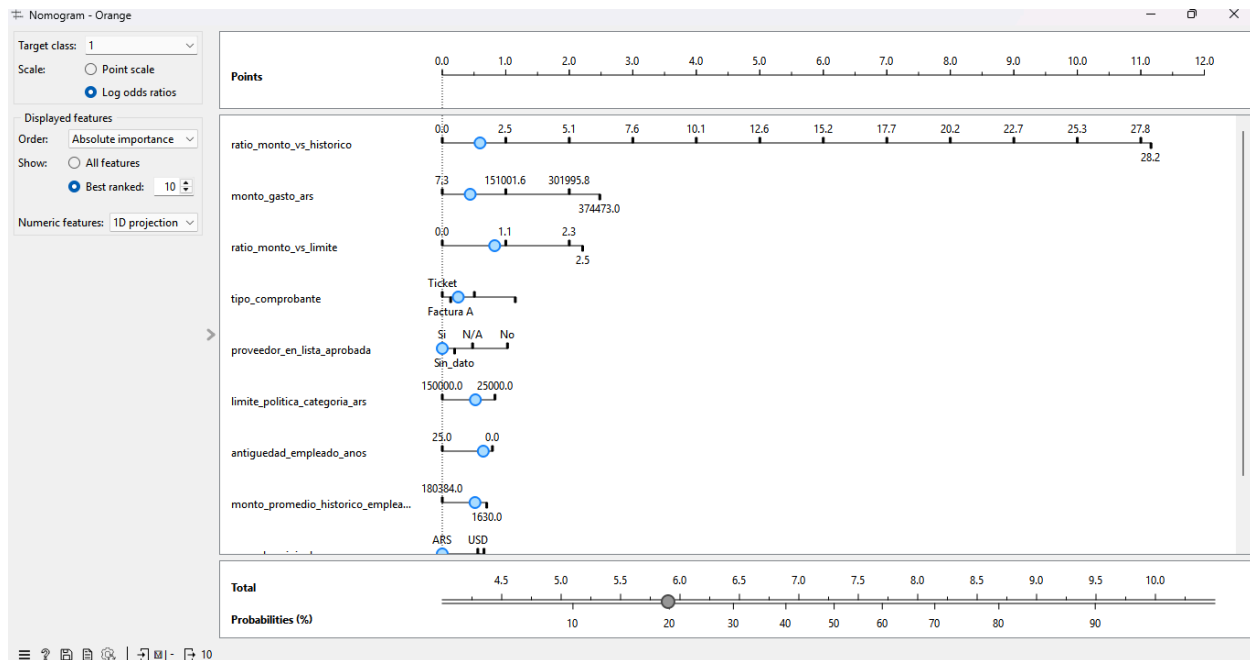
- Las dos variables derivadas son las más predictivas: ratio_monto_vs_limite y ratio_monto_vs_historico ocupan el primer y segundo lugar. En conjunto concentran aproximadamente el 71% de la importancia del modelo. Este es el resultado más relevante para validar la decisión de hacer Feature Engineering: las variables creadas en

la etapa de preprocesamiento (Sección 3.3) son MÁS predictivas que cualquier variable cruda del dataset.

- **Variables de control y categóricas en segundo nivel:** proveedor_en_lista_aprobada y tipo_comprobante_num (la versión ordinal que creamos en Formula) aparecen en posiciones 3 y 4. Esto valida la decisión de codificar tipo_comprobante como ordinal (y no como One-Hot).
- **Variables temporales con importancia cero:** hora_carga_sistema, dia_semana_gasto, dias_entre_gasto_y_rendicion y flag_fin_de_semana tienen importancia prácticamente nula. Esto confirma la refutación de la hipótesis H3.
- **Nivel jerárquico sin importancia:** tanto nivel_jerarquico_empleado como nivel_jerarquico_num tienen importancia cero. Es un hallazgo contraintuitivo: una pensaría que los empleados de mayor jerarquía (con mayor poder de gasto) podrían concentrar más casos de fraude. Los datos refutan esa hipótesis: el fraude se distribuye por igual entre todos los niveles.

6.2 Nomogram (interpretación de Logistic Regression)

Aunque Logistic Regression no es el modelo elegido como ganador, su widget Nomogram aporta interpretabilidad valiosa: permite ver el peso direccional (y la dirección del efecto) de cada variable sobre la probabilidad de fraude.



Lectura del Nomogram

- **ratio_monto_vs_historico:** rango de 0 a 27.8 puntos. Es la variable con MAYOR rango de peso individual. Confirma que el desvío respecto del histórico del empleado es la señal más fuerte.

- **ratio_monto_vs_limite:** rango de 0 a 2.5 puntos. Aporta de manera consistente.
- **tipo_comprobante:** Factura A asigna bajo riesgo (puntos cercanos a 0), mientras que Sin Comprobante asigna alto riesgo (puntos hacia el extremo superior). La direccionalidad es la esperada.
- **proveedor_en_lista_aprobada:** 'Si' aporta 0 puntos, 'No' aporta el máximo. Direccionalidad consistente con H2.
- **antigüedad_empleado_anos (hallazgo inesperado):** el rango de la variable es 25.0 a 0.0 (invertido). Esto significa que a MENOR antigüedad, MAYOR contribución a la probabilidad de fraude. Es un hallazgo contraintuitivo importante: los empleados más nuevos tendrían mayor propensión, no los más antiguos.

7. Conclusiones y recomendaciones

7.1 Conclusiones principales

El proyecto desarrolló un modelo de clasificación binaria operativo para asistir al equipo de Auditoría Interna en la priorización de rendiciones de gasto. Las conclusiones más relevantes son:

- **Modelo final operativo:** Gradient Boosting con AUC 0.888, Recall 0.694, Precision 0.844 sobre el test set. El modelo es capaz de capturar el 69% de los fraudes con sólo 16% de falsas alarmas, lo cual lo convierte en una herramienta directamente integrable al flujo de trabajo de auditoría.
- **El Feature Engineering fue decisivo:** las dos variables derivadas (ratios) concentran el 71% de la importancia del modelo y multiplican por varias veces la capacidad discriminativa que tendrían sus componentes individuales. Es la pieza más valiosa del pipeline.
- **Hallazgos contraintuitivos:** el comportamiento temporal NO es predictivo (refutación de H3), y el nivel jerárquico del empleado tampoco aporta información. La señal de fraude está mayormente en variables monetarias relacionales (ratios) y de control (proveedor, comprobante).

7.2 Recomendaciones operativas

A partir de los resultados del modelo, recomendamos las siguientes acciones concretas:

Recomendación 1 - Implementación en producción con cola priorizada

Implementar el modelo como un servicio que califique cada nueva rendición y derive a Auditoría sólo aquellas con probabilidad de fraude superior al umbral de 0.5. Esto permitiría a Auditoría concentrarse en el 33% del volumen capturando el 69% del fraude real, con una eficiencia tres veces superior al proceso manual actual.

Recomendación 2 - Reglas duras complementarias para los Falsos Negativos

El modelo deja escapar el 31% de los fraudes (417 sobre 1.362 en el test). Para mitigar este riesgo, recomendamos COMPLEMENTAR el modelo con reglas duras de negocio para casos de alto monto absoluto, independientemente de la predicción del modelo. Por ejemplo: todo gasto que exceda \$300.000 ARS debería ir a auditoría obligatoria, sin pasar por el filtro del modelo. Esto cubre la cola larga de la distribución donde el modelo puede tener menor confianza estadística por baja densidad de ejemplos.

Recomendación 3 - Mejora de los datos en el proceso de carga

Dado que la variable proveedor_en_lista_aprobada resultó ser una de las top 5 más predictivas, y dado que actualmente tiene un 3% de nulos (que tratamos como 'Sin_dato'), recomendamos reforzar el proceso de carga del sistema para volver este campo OBLIGATORIO al ingresar una

rendición. Esto eliminaría la ambigüedad de la categoría 'Sin_dato' y mejoraría la calidad del modelo.

7.3 Limitaciones del trabajo y mejoras futuras

- La aplicación de SMOTE antes del split train/test introduce un sesgo metodológico menor que podría corregirse implementando el pipeline en scikit-learn con SMOTE actuando sólo sobre el train.
- Los hiperparámetros de los modelos se mantuvieron en los defaults de Orange. Un proceso de tuning (Grid Search o Random Search) podría mejorar el rendimiento marginalmente.
- El modelo se evaluó sobre un único split 80/20. Un esquema más robusto sería repetir el entrenamiento con varios splits aleatorios (replicable random sampling) y reportar media $\pm$ desvío de cada métrica.
- La interpretabilidad de Gradient Boosting es limitada. Para entender decisiones individuales se podría incorporar SHAP values en una implementación productiva.

Anexo - Código Python de SMOTE

Por completitud, se incluye el código exacto utilizado en el widget Python Script para la implementación de SMOTE 60/40. El código no utiliza la librería externa imblearn sino una implementación manual basada en scipy y numpy, que son las librerías disponibles dentro de Orange.

```
import numpy as np
from Orange.data import Table

# === CONFIGURACIÓN ===
TARGET_MINORITY_RATIO = 0.40 # 40% para la clase minoritaria

data = in_data

if data is None or data.domain.class_var is None:
    out_data = data
else:
    X = data.X
    y = data.Y
    M = data.metas

    classes, counts = np.unique(y, return_counts=True)

    majority_class = classes[np.argmax(counts)]
    minority_class = classes[np.argmin(counts)]
    n_majority = counts.max()
    n_minority = counts.min()

    # Cálculo del tamaño objetivo de la clase minoritaria
    n_total_target = int(n_majority / (1 - TARGET_MINORITY_RATIO))
    n_minority_target = int(n_total_target * TARGET_MINORITY_RATIO)
    n_to_generate = n_minority_target - n_minority

    if n_to_generate <= 0:
        out_data = data
    else:
        # SMOTE: K-vecinos + interpolación
        X_minority = X[y == minority_class]
        k = 5
        n_minority_total = len(X_minority)

        np.random.seed(42)

        from scipy.spatial.distance import cdist
        distances = cdist(X_minority, X_minority)
        neighbors_idx = np.argsort(distances, axis=1)[: , 1:k+1]

        synthetic_X = np.zeros((n_to_generate, X.shape[1]))

        for i in range(n_to_generate):
            idx_base = np.random.randint(n_minority_total)
            base_point = X_minority[idx_base]
            idx_neighbor = neighbors_idx[idx_base][np.random.randint(k)]
            neighbor_point = X_minority[idx_neighbor]
            alpha = np.random.random()
            synthetic_X[i] = base_point + alpha * (neighbor_point - base_point)

        synthetic_y = np.full(n_to_generate, minority_class)

        # Metas: identificadores ficticios para los sintéticos
```

```
n metas = M.shape[1] if M.ndim > 1 else 1
synthetic_M = np.array(
    [[f"SYNTH_{i:05d}"] * n metas for i in range(n_to_generate)],
    dtype=object
)

new_X = np.vstack([X, synthetic_X])
new_y = np.concatenate([y, synthetic_y])
new_M = np.vstack([M, synthetic_M])

out_data = Table.from_numpy(data.domain, new_X, new_y, new_M)
```